高级语言程序设计期末大作业报告

报告名称：孔明棋

专业：数据科学与大数据技术（精英班）

班级：工科 6 班

学号：2454259

姓名：费凡

完成日期：2025.6.9

一、项目简介

本次大作业是基于 C++和 EasyX 图形库开发的孔明棋小游戏。孔明棋也叫做法国独立钻石棋（Peg Solitaire），是一种智力游戏，通常由一个棋盘和若干颗棋子组成。最经典的 Solitaire 棋盘在一个 33 孔的盘面上摆放 32 颗棋子。（初始最中间位置不放子）游戏目标是通过跳跃吃子的方式，最终只剩下一颗棋子，并且这颗棋子最好位于棋盘的中心位置。游戏规则如下：每次选定一颗棋子，跳过直接相邻的一颗棋子，并放到空的孔里，被跳过的棋子被移出棋盘。每次移动可以是上下左右四个方向，但不能是斜向。如果盘面上只剩一颗棋子则胜利，如果剩下多颗棋子而且无棋子可移动则失败。

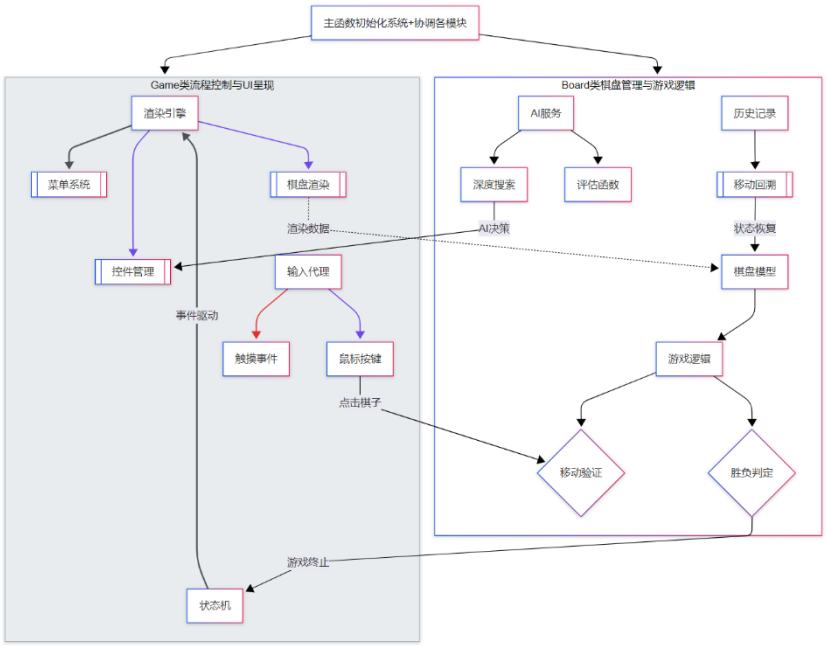
二、设计思路

1.基本内容设计思路

本程序采取面向对象的设计方法，主要分成两个类：Board 类和 Game 类。其中 Board 类用来负责棋盘状态的管理与游戏逻辑的实现，Game 类用来控制游戏流程的控制及实现与用户游戏界面的可视化呈现。

在 Board 类中，通过定义基本布局模式及设计残局模式实现棋盘布局的管理，通过定义绘制函数绘制棋盘和棋子，并利用 7*7 的二维数组实时更新棋盘的状态；通过添加历史记录的栈实现悔棋的功能；通过添加走棋规则函数实现游戏中实际走棋子移动的处理；通过添加重置函数实现重新开始功能；采用 AIMoveHint 结构存储 AI 推荐的移动；并在最后添加了游戏胜利或失败的 bool 值判断。

在 Game 类中，首先对在全局空间定义的枚举类型中不同游戏状态进行定义和管理，并对游戏进程中的不同按钮进行定义；接着对鼠标点击与按钮交互进行了处理，便于进行不同按钮点击时的不同功能的代码实现；最后便是关键的游戏流程实现以及功能切换。同时在 Game 类的最后还加入了 UI 界面绘制及相关文字提示。



2.加分项实现

（1）**更美观的界面**

通过 easyx 图形库实现了完整的图片背景实现，从初始的普通颜色，到后来渐变颜色的想法，再到后来插入我喜欢的图片作为背景。

①渐变色实现：不同坐标位置采取不同的颜色，产生渐变效果，带来好的视觉体验。

关键代码： int R = 220 - (40 * y) / HEIGHT; int G = 240 - (30 * y) / HEIGHT; int B = 255 - (15 * y) / HEIGHT;后面采取 easyx 相关库函数设置颜色等。

②背景图片插入：关键代码： 《1》IMAGE backgroundImg; // 导入背景图片对象

《2》void init() { initgraph(WIDTH, HEIGHT);

setbkcolor(WHITE); loadimage(&backgroundImg, _T("jidi.jpg"));} // 加载背景图片

《3》void drawBackground() const{ putimage(0,0,&backgroundImg);}//绘制图片背景

③独特的主题颜色设计：棋盘选取木质纹理的格子背景，搭配浅棕色棋盘格子和红棋子（默认状态），绿色棋子（选中状态），橙色棋子（AI 提示的待移动棋子）黄色小棋子（AI 提示目标位置），整体颜色协调，对比度适宜，视觉效果较好，在鼠标悬浮在按钮上时产生颜色变化以示区分。

（2）**更复杂的规则**

代码中实现了五种棋盘的设计：经典模式+4 种残局模式，代码类似，也实现了完整的残局选择界面，用户可以选择任何一个残局进行游戏或者返回上一个经典模式与残局模式选择的界面，选择及切换灵活。

（3）**更智能的逻辑**

设计了多级 AI 智能走棋算法，首先利用深度优先算法（DFS）预先计算最优解路径，接着设定跟随机制，玩家可以从头根据这个最优解提示完成通关，在游戏过程中若使用 AI 提示，系统会自动判断是否符合前面计算的最优解路径，符合则会按照路线来，不符合再转向反应更加迅速的 A*算法，根据用户选择实时计算最佳移动，防止卡死（全局搜索需要大量时间及 CPU 内存消耗）。

设计了综合评估函数来指导 AI 决策，基础目标是剩余棋子越少越好，可移动棋子越多越好，通过设置中心权重鼓励 AI 将棋子移向中心，通过得分机制实时评估目前走棋状态并做出实时调整。

实际写代码过程中我也经历了反复的尝试，从一开始的最基本的 AI 随机移动算法处理棋子移动，但最后结果不甚理想（剩余棋子基本都 > 6 个），到使用较高级的 alpha-beta 剪枝算法，发现不易实现且效果仍不令人满意（剩余棋子>5 个），接着转向 A*算法，最终剩余棋子>4,然后探索发现了深度优先算法（DFS），可以使全盘从头到尾进行

AI 提示完成求解!! 这让我非常激动，但很遗憾的是发现走棋几步后再进行提示会出现卡死的情况，查阅资料后发现这是由于中间会有大量的走棋分支变化导致全局深度优化实现过程极慢，也极大地影响游戏体验，在沮丧的同时，我又想到了，假如在开局调用这个 DFS 算法后面调用反应迅速的 A*算法会不会有奇效？真的是呢!! 这样不仅解决了卡死问题，也能保证开局能顺利解题。与此同时，我还想到了进一步的优化方式，在一开始预先计算最优路径，接着设定跟随机制，在中间阶段即使不完全符合最优路径，但只要有满足的走棋部分就可以进行模仿，实际操作下来不仅迅速，而且在开局基本乱点的情况下也能基本确保剩余步数<=3 步!! 至此完成了最终的 AI 智能走棋版本。

上述的思考花费了我两天的时间，但我觉得这些思考真的是值得的!! 不仅是完善了 AI 走棋逻辑，也让我对各种算法及优化方式的理解也更加深入了!!

三、主要功能实现

1.不同游戏模式

（1）33 格 32 子的经典模式

（2）四个残局模式可供选择（均有解）

2.完整游戏功能

（1）棋子选择与移动

（2）悔棋

（3）AI 智能提示走棋系统

（4）胜负判定

（5）重新开始

（6）游戏过程中可以实时返回菜单

（7）菜单栏里有游戏规则介绍

3.用户友好型界面

（1）直观的图形化棋盘

（2）带有视觉冲击力的背景图片

（3）响应式的按钮设计（鼠标放在按钮上有颜色变化）

（4）实时显示游戏状态（如剩余棋子和输赢状态）

四、开发过程中遇到的问题及解决方法

1.画面卡顿的问题与兼顾系统 CPU 消耗的矛盾：开始设计 Sleep（100）尽力减少 CPU

消耗，但发现棋子移动画面非常卡顿，于是改为 Sleep（1）实现短暂“休息”功能，在保证画面质量的前提下也减少了 CPU 消耗。

2.棋盘闪烁问题：采用双缓冲绘图技术，通过将所有的绘制操作先绘制到内存缓冲区，然后一次性刷新到屏幕，有效消除了闪烁现象。

3.悔棋功能消耗大量内存：设计轻量级的结构体记录移动过程与结构，并使用栈结构保存历史记录，大大提高内存效率。

4.游戏功能增加导致的游戏状态管理困境：采用状态机模式管理游戏流程，开头在全局位置事先定义好枚举变量，便于后续直接调用，逻辑更加清晰，也明确了定义状态和转换规则，使代码更清晰、更易维护。

五、智慧编程工具的作用

1.在遇到某些专业知识的短缺时，比如不清楚具体颜色对应的相关特征数值，不需要再查询相关知识，直接询问 copilot 就可以让它模仿某个书写格式，直接满足相关绘图的要求，方便快捷。

2.在遇到代码组织结构混乱时，可以让 copilot 帮忙整合代码结构，并帮助我完善代码细节，可以帮我完成某些重复性的代码模仿补全。

3.当发现某一部分很难处理时，比如关于 AI 提示中算法的处理，我给他提供我的想法，它能根据我的需求给出相应的代码提示供我参考，帮助我完成 AI 智能走棋这一复杂的工程。

六、心得体会

1. 通过本项目，我深刻理解了面向对象设计原则的实际应用，并体会到了它的重要意义。对于我的代码，Board 类只负责棋盘及逻辑，Game 类只负责界面和流程，这样当我修改走棋逻辑时，只需要在 Board 类中相应函数地方进行修改，而不需要全局操作，而如果想把界面变美观，只需要在 Game 类操作即可，这样的程序架构更易于维护，也更方便进行修改操作。而且可以通过类的封装功能隐藏棋盘内部状态，通过巩固方法提供访问。进行类的设计也可以将游戏功能划分为独立模块，便于开发和测试。

2. 通过本项目，我对算法设计的理解更加深入，我能够规划出项目的大框架，利用一个 7*7 的二维数组来表示棋盘有无棋子的状态，大大简化了研究问题；同时，我能清晰理解到栈的存储对实现悔棋功能有重要帮助，利用这个存储功能，我还想到了在 AI 智能走棋逻辑设置时，让 AI 记录最优解，并在之后利用这个存储的结果进行后续流程的进行，最后取得游戏胜利，在不同算法的博弈中我最终也实现了一个比较合理且可行的 AI 走棋算法，可以有效平衡性能与走棋复杂度，更好接近最优解法。

3. 通过本项目，我对用户界面的设计有了全新认知，作为一个游戏设计师，提供给用户的一定是最优质的游戏体验感，所以在其中添加选中棋子的高亮与颜色变化及移动到的

位置的突出显示是非常必要的。同时，为了让用户在菜单中能够更好确认鼠标所在位置，对鼠标是否悬停在菜单位置进行了颜色区分，并在之后提供实时的清晰游戏状态提示。整个游戏界面颜色的设置也有高对比度体现，棋子采用大尺寸，可以提供更加优质的游戏体验

同时，为了保证游戏过程中的体验感，当用户选择到了不可移动的棋子不会出现弹窗并报错，而是保持静默，并让棋子保持高亮状态，让用户能够清楚看到目前走棋的不合理性并做出改变，棋子切换的流程也非常丝滑，直接选中想选的棋子即可，而不再需要先取消选择再选下一个棋子。同时，游戏状态也在系统中实现了自动保存，方便用户实时进行悔棋与操作。

4. 游戏攻略：分为经典模式和残局模式，残局 1-4 难度逐渐加深，用户可以根据情况自行选择。进入游戏之后用户按照游戏规则进行操作，可以从开头就进行 AI 提示，AI 能够保证你走向最终胜利，在游戏过程中，你也可以使用 AI 提示给你一个较合理的思路（虽然不一定是最后完成的最优解），合理使用 AI 提示及选定使用时机是在没有掌握走棋规律之前取得成功的关键！同时，你也可以实时使用悔棋来回顾你之前的操作，寻找问题，相信你一定能成功！

5. AI 走棋逻辑设定的心路历程（见 P3-4）

七、源代码组织结构

1.文件结构：头文件引入→常量定义→枚举类型定义→结构体定义→Board 类的实现→Game 类的实现→main 函数调用

2.关键代码片段

走棋逻辑的制定（部分）

```
private:
    // 尝试移动棋子
    bool tryMove(int fromX, int fromY, int toX, int toY) const
    {
        if (state[toX][toY] != 0)
        {
            return false;
        }

        if (fromX == toX) //检查横向是否能移动
        {
            int distance = abs(fromY - toY);
            if (distance != 2)
            {
                return false;
            }

            int midY = (fromY + toY) / 2;
            if (state[fromX][midY] != 1)
            {
                return false;
            }
        }
        else if (fromY == toY)
        {
            int distance = abs(fromX - toX);
            if (distance != 2)
            {
                return false;
            }

            int midX = (fromX + toX) / 2;
            if (state[midX][fromY] != 1)
            {
                return false;
            }
        }
        else
        {
            return false;
        }

        return true;
    }
```

关键游戏界面实现框架

```
public:
    void init()
    {
        initgraph(WIDTH, HEIGHT);
        setbkcolor(WHITE);
        loadimage(&backgroundImg, _T("jidi.jpg")); // 加载背景图片
    }

    void run()
    {
        BeginBatchDraw();//设置双缓冲绘图模式，减少闪烁

        while (true)
        {
            cleardevice();
            drawBackground();//在游戏开始前绘制背景

            switch (currentState)
            {
            case MENU:
                showMainMenu();
                break;
            case MODE_SELECTION:
                showModeSelection();
                break;
            case PLAYING:
                playGame();
                break;
            case SHOWING_RULES:
                showRules();
                break;
            case GAME_OVER:
                showGameOver();
                break;
            }

            FlushBatchDraw();// 刷新屏幕
            handleMouseInput();
            Sleep(1);// 减少CPU占用
        }

        EndBatchDraw();// 结束双缓冲
    }
```

```
public:
    // 判断是否胜利
    bool isWin() const
    {
        return remainingPieces == 1;
    }

    // 判断是否失败
    bool isLose() const
    {
        if (remainingPieces <= 1) return false;

        for (int i = 0; i < 7; ++i)
        {
            for (int j = 0; j < 7; ++j)
            {
                if (state[i][j] == 1)
                {
                    if (canMove(i, j, i - 2, j) || canMove(i, j, i + 2, j) || canMove(i, j, i, j - 2) || canMove(i, j, i, j + 2))
                    {
                        return false;
                    }
                }
            }
        }
        return true;
    }
```

游戏胜利或失败的判断

```
//UI界面绘制
private:
    void showMainMenu()
    {
        // 绘制标题
        settextstyle(48, 0, _T("宋体"), 0, 0, FW_BOLD, false, false, false);
        settextcolor(RGB(70, 130, 180));
        outtextxy(WIDTH / 2 - 75, 100, _T("孔明棋"));

        //绘制副标题
        settextstyle(24, 0, _T("宋体"));
        settextcolor(RGB(100, 100, 150));
        outtextxy(WIDTH / 2 - 85, 180, _T("Peg Solitaire"));

        // 绘制菜单选项
        settextstyle(36, 0, _T("宋体"));

        // 开始游戏按钮
        setfillcolor(hoveredOption == CLASSIC_MODE ? RGB(100, 200, 100) : RGB(180, 230, 100));
        fillrectangle(startButton.left, startButton.top, startButton.right, startButton.bottom);
        settextcolor(RGB(0, 0, 0));
        outtextxy(WIDTH / 2 - 80, 260, _T("开始游戏"));

        // 游戏规则按钮
        setfillcolor(hoveredOption == GAME_RULES ? RGB(100, 180, 255) : RGB(180, 200, 255));
        fillrectangle(rulesButton.left, rulesButton.top, rulesButton.right, rulesButton.bottom);
        settextcolor(RGB(0, 0, 0));
        outtextxy(WIDTH / 2 - 80, 330, _T("游戏规则"));

        // 退出游戏按钮
        setfillcolor(hoveredOption == EXIT_GAME ? RGB(255, 150, 150) : RGB(255, 180, 180));
        fillrectangle(exitButton.left, exitButton.top, exitButton.right, exitButton.bottom);
        settextcolor(RGB(0, 0, 0));
        outtextxy(WIDTH / 2 - 80, 400, _T("退出游戏"));
    }

    void showModeSelection()
    {
        // 绘制标题
        settextstyle(36, 0, _T("宋体"), 0, 0, FW_BOLD, false, false, false);
        settextcolor(RGB(70, 130, 180));
        outtextxy(WIDTH / 2 - 100, 150, _T("选择游戏模式"));
```

UI 界面渲染（部分）

```
// 优化后的评估函数 - 启发式函数
int evaluateBoard() const
{
    // 剩余棋子越少越好，可移动棋子越多越好，中心权重高
    int score = remainingPieces * 100;

    // 统计可移动棋子数
    int movable = 0;
    for (int i = 0; i < 7; ++i)
        for (int j = 0; j < 7; ++j)
            if (state[i][j] == 1)
                if (canMove(i, j, i - 2, j) || canMove(i, j, i + 2, j) ||
                    canMove(i, j, i, j - 2) || canMove(i, j, i, j + 2))
                    ++movable;
    score -= movable * 10;

    // 中心权重（鼓励棋子靠近中心）
    static const int centerWeight[7][7] =
    {
        {0,0,1,2,1,0,0},
        {0,1,2,3,2,1,0},
        {1,2,4,5,4,2,1},
        {2,3,5,8,5,3,2},
        {1,2,4,5,4,2,1},
        {0,1,2,3,2,1,0},
        {0,0,1,2,1,0,0}
    };
    int centerScore = 0;
    for (int i = 0; i < 7; ++i)
        for (int j = 0; j < 7; ++j)
            if (state[i][j] == 1)
                centerScore += centerWeight[i][j];
    score -= centerScore * 2;

    return score;
}
```

基本 AI 走棋算法设计（部分）